

Securing SME Agent Skills via Fine-Tuned SLM Proxies

A dissertation submitted in partial fulfilment of the
requirements for the degree of Master of Science.

by Benin Mukabanah

Masters Degree in Computing and Information Technology

Cardiff School of Technologies

Cardiff Metropolitan University, Cardiff

July 2026

Declaration

I hereby declare that this dissertation entitled '*Securing SME Agent Skills via Fine-Tuned SLM Proxies*' is entirely my own work and it has never been submitted nor is it currently being submitted for any other degree.

Candidate

Signature:

Name: Benin Mukabanah

Date:

Director of Studies

Signature:

Name: Dr P Jenkins

Date:

ABSTRACT

Insert Here.....

ACKNOWLEDGEMENTS

The author would like to thank Dr P Jenkins for invaluable supervision and guidance throughout this project. Gratitude is also extended to the staff of the Cardiff School of Technologies and the Cardiff Metropolitan University library for their continued support.

CONTENTS

1	Introduction	1
1.1	Overview and Background	1
1.2	Aims and Objectives	2
1.3	Approach Used	2
1.4	Assumptions	2
1.5	Limitations and Achievements	3
1.6	Overview of the Dissertation Structure	3
2	Literature Review	5
2.1	The SME Operational Paradox and Digital Adoption	5
2.2	The Evolution of Agentic AI: MCP vs. Agent Skills	6
2.2.1	Key Differences and Architectural Analysis	7
2.3	The Security Threat: Indirect Prompt Injections (IPI) in Tool-Calling	8
2.4	SLMs as Efficient and Secure Proxies	9
2.5	Research Gaps	10
2.6	Summary	11
3	Methodology	12
3.1	Research Strategy: Design Science Research	12
3.2	Data Generation: Programmatic Matrix Synthesis	13
3.2.1	Domain Category Mappings	15
3.2.2	Shortcut Learning Mitigation	16
3.2.3	Benign Hard Negatives	16
3.2.4	Dataset Dimensions and Outputs	17
3.3	Artifact Construction: Supervised Fine-Tuning (SFT) and LoRA	18
3.3.1	SFT Dataset Structure and Task Formulation	19
3.3.2	Dataset Partitioning	20
3.4	Academic Justifications	21
3.4.1	Justification for the Partitioning Ratio (SFT vs. Holdout)	21
3.4.2	Justification for the Deterministic Split (Fixed Seed = 42)	21
3.4.3	Justification for the Dual-Mode Setup (Standard vs. Domain-Aligned)	22
3.5	Evaluation Framework	22
3.5.1	Evaluation Execution Pipeline	23
3.6	Summary	26
4	Results	27
4.1	Quantitative Findings	27
4.2	Qualitative Findings	28
5	Discussion	29
5.1	Quantitative Results	29
5.2	Qualitative Themes	29
5.3	Limitations	29

6 Conclusions	30
---------------------	----

LIST OF TABLES

Table 1	Architectural and Functional Contrasts Between MCP and Agent Skills	7
Table 2	Taxonomy of Evaluated Indirect Prompt Injection (IPI) Attack Vectors	15
Table 3	Composition of the programmatic synthesis evaluation dataset	18
Table 4	Dataset Partitioning Splits for SFT Training and Holdout Evaluation	20
Table 5	Accuracy results by experimental condition ($n = 30$)	27

LIST OF FIGURES

Figure 1	Systematic Execution Flow of the SLM Proxy Assessment Harness	25
Figure 2	Distribution of accuracy scores by condition	27

INTRODUCTION

Overview and Background

Approximately 90% of all businesses and more than 50% of all jobs globally are small and medium-sized firms (SMEs), which serve as the foundation of the global economy (Sánchez et al. 2025). These businesses urgently need to implement Artificial Intelligence (AI) to automate repetitive operations, optimise workflows, and boost operational productivity in order to stay competitive in an increasingly digital economy (Ode 2026). However, SMEs often face an “Operational Paradox”: they lack the substantial financial resources, specialised IT infrastructure, and technical staff needed to run or administer big, general-purpose AI models (Sánchez et al. 2025), despite their dire need for automation to grow.

AI has recently progressed beyond passive conversational chatbots to self-governing “agents” that can communicate with external surroundings and systems (Zhan et al. 2024). The Model Context Protocol (MCP), which serves as a global “USB-C port” for AI applications, was developed to standardise how AI accesses external tools (Whittaker 2026). SMEs are increasingly depending on a lightweight substitute called **Agent Skills** (Whittaker 2026), whereas big IT firms are capable of managing complicated MCP server infrastructure. The AI receives reusable procedural knowledge and commands via Agent Skills, which are standardised, version-controlled directories anchored by a `SKILL.md` file (Agent Skills 2026). Agents load these skills using “progressive disclosure,” reading only the most basic metadata at first and only expanding the entire instructions into their context window when the task (Agent Skills 2026) specifically requests it.

Agent skills present two significant structural hurdles for SMEs, notwithstanding their enormous utility. Initially, they make these integrations extremely vulnerable to **Indirect Prompt Injection (IPI) attacks** (Gulyamov et al. 2026). In order to deceive the AI into carrying out unauthorised operations, such as exfiltrating private data (Zhang et al. 2025), attackers can include malicious commands within the Markdown body or YAML frontmatter of a `SKILL.md` file using strategies like hidden Unicode tags or out-of-scope parameters. Second, the “Context Tax”: a computational

penalty where inflated contextual history causes the model to consume enormous amounts of expensive tokens without enhancing task performance (Fraser and Lindenbauer 2025). This is exacerbated by the unstructured and verbose nature of custom SME instructions.

Aims and Objectives

The primary aim of this research is to design, construct, and evaluate a fine-tuned Small Language Model (SLM) proxy that secures SME workflows by auditing and compressing `SKILL.md` files, protecting them against Indirect Prompt Injections (IPIs) and mitigating the “Context Tax.”

To achieve this overarching aim, the following objectives have been established:

1. **Synthesize** a targeted evaluation corpus yielding 3,276 test instances across two modes (comprising 1,054 unique standard and 414 unique domain-aligned attack scenarios, each evaluated across Base and Enhanced settings, alongside 170 benign hard negative controls), representing 2,278 unique virtual test cases in total, operating as an in-memory JSON pipeline based on established prompt injection taxonomies (Zhan et al. 2024).
2. **Fine-tune** the Gemma 4 E2B SLM via Supervised Fine-Tuning (SFT) using Low-Rank Adaptation (LoRA) to successfully classify security risks (Security Audit) and distill skill metadata (Semantic Distillation) (Unsloth 2026a).
3. **Evaluate** the proxy’s performance by quantitatively measuring its Security Efficacy via the Attack Success Rate (ASR-valid), format integrity, and Token Compression Ratio.

Approach Used

This dissertation adopts a **Design Science Research (DSR)** methodology, which focuses on constructing and rigorously evaluating a functional artifact to solve a practical, real-world problem (Zupan 2025). The core artifact is a secure middleware proxy powered by Google DeepMind’s **Gemma 4 E2B**, a 2-billion parameter Small Language Model optimized for tool use and long-context workflows (Unsloth 2026b).

The approach utilizes Parameter-Efficient Fine-Tuning (PEFT) via LoRA within the Unsloth framework. This method ensures the deployed model can be quantized to 4-bit precision, allowing it to run locally on standard consumer hardware with just 5GB of RAM: directly aligning with SME hardware constraints (Unsloth 2026b). The proxy is trained to perform two automated passes on custom `SKILL.md` files before they reach downstream operational AI agents: Pass A to flag and neutralize IPI attacks (Zhang et al. 2025), and Pass B to actively compress verbose metadata to eliminate context bloat (Fraser and Lindenbauer 2025).

Assumptions

This research is based on several foundational assumptions:

- **Adoption Trends:** It assumes that SMEs will increasingly adopt lightweight, structured Agent Skills (`SKILL.md` files) to grant specialized knowledge to LLMs, rather than hosting complex, heavy MCP servers themselves (Whittaker 2026).
- **Adversarial Threat Models:** It assumes that malicious actors will utilize IPI techniques targeting structured metadata, such as preference manipulation and hidden obfuscation, mirroring the attack taxonomies defined by (Zhan et al. 2024) and (Zhang et al. 2025).
- **Synthetic Data Validity:** Given the scarcity of real-world databases containing poisoned SME tools, this work assumes that a synthetic dataset constructed via Frontier Model Distillation and the INJECAGENT combinatorics methodology accurately reflects potential real-world adversarial threats ((Zhan et al. 2024); (Unsloth 2026b)).

Limitations and Achievements

Limitations: Because the dataset is entirely synthesized, the proxy’s defensive capabilities are intrinsically bound to the specific threat models explicitly documented in the MSB and INJECAGENT taxonomies ((Zhan et al. 2024); (Zhang et al. 2025)). As AI hacking techniques evolve rapidly, the fine-tuned model may struggle against entirely novel, zero-day injection paradigms without subsequent retraining (Gulyamov et al. 2026). Furthermore, the evaluation is primarily quantitative (calculating automated success and pass rates) and does not deploy the artifact into a live, active SME operational environment for longitudinal study.

Achievements: Despite these limitations, this research is expected to deliver a highly practical, deployable solution to a pressing industry vulnerability. By proving that a 2-billion parameter SLM can successfully filter complex IPI attacks and optimize context usage, this study challenges the prevailing assumption that massive LLMs are strictly required for reliable context routing and security screening (Jhandi et al. 2026). It actively bridges the gap between SME resource limitations and critical AI security needs.

Overview of the Dissertation Structure

The remainder of this dissertation is organized as follows:

- **Chapter 2: Literature Review:** Critically examines the current landscape of AI adoption in SMEs, the evolution from basic LLMs to MCP tool-calling agents, the mechanics of Indirect Prompt Injections, and the efficacy of SLMs in handling specialized tasks.
- **Chapter 3: Methodology:** Details the Design Science Research (DSR) strategy, the programmatic matrix synthesis of the 3,276-item evaluation corpus, and the Parameter-Efficient Fine-Tuning (PEFT) framework using Low-Rank Adaptation (LoRA).
- **Chapter 4: Results:** Presents the quantitative and qualitative findings of the experimental evaluation.
- **Chapter 5: Discussion:** Interprets the findings, contextualizes them within the literature, and discusses the limitations of the research design.

- **Chapter 6: Conclusions:** Summarizes the research achievements, addresses each objective in turn, and proposes directions for future work.

LITERATURE REVIEW

This chapter critically examines the current literature surrounding the operationalisation, security, and optimisation of artificial intelligence within Small and Medium-sized Enterprises (SMEs). Before delving into the architectural transition from monolithic Large Language Models (LLMs) to tool-integrated Agentic AI via the Model background Protocol (MCP) and Agent Skills, the broader background of the SME “Operational Paradox” is established. The article then assesses the growing body of research on the use of optimised Small Language Models (SLMs) as effective, secure middleware solutions and critically examines the serious security flaws brought about by these architectures, including Indirect Prompt Injections (IPIs).

The SME Operational Paradox and Digital Adoption

Approximately 90% of all businesses and more than 50% of all jobs globally are small and medium-sized firms (SMEs), which form the foundation of the global economy (Sánchez et al. 2025). SMEs are realising that artificial intelligence (AI) is a strategic necessity to boost automation, improve operational efficiency, and spur growth in order to stay competitive in an increasingly digital environment. This quick adoption is supported by recent empirical data; according to a 2026 report on UK businesses, 35% of SMEs are currently actively adopting AI technology—a significant increase from the previous year—mainly to expedite daily operations, carry out research, and manage administrative duties (Lewis 2026). Similarly, regional research shows that 65% of SMEs use AI for data analysis and content production (Ode 2026).

Nevertheless, significant systemic weaknesses are hidden by these surface-level adoption measurements. SMEs are aggressively experimenting with AI, but they are unable to operationalise the technology to produce revolutionary outcomes due to significant structural constraints. According to empirical data, substantial knowledge gaps, which were mentioned by 77% of SME respondents, as well as severe time, resource, and financial constraints, are the main barriers to deep AI integration (Ode 2026). Additionally, SMEs often have dispersed data silos, disjointed legacy systems, and

lax security and identity restrictions, all of which make it extremely difficult to deploy autonomous AI in a safe and dependable manner (Lewis 2026).

The “Operational Paradox” (Sánchez et al. 2025) is the result of these compounding limitations. On the one hand, in order to overcome resource constraints, scale operations, and endure in a market that is changing quickly, SMEs desperately need AI-driven automation. However, they inherently lack the funding, sophisticated technology infrastructure, and skilled workers needed to securely host, deploy, or secure large AI models (Sánchez et al. 2025). Because SMEs have smaller profit margins and less absorptive capacity than major multinational corporations, the high upfront costs and intricate upkeep of reliable AI systems are both technically and financially prohibitive (Sánchez et al. 2025).

As a result, SMEs and larger tech-enabled businesses are seeing a growing digital divide (Lewis 2026). Many SMEs are left depending on disjointed, superficial tools that are unable to connect with one another, whereas highly resourced corporations are completely redesigning their operational models with secure, integrated AI architectures that save entire weeks of manual effort (Lewis 2026). SMEs are at a serious, compounding competitive disadvantage in the digital economy because of this operational imbalance, which frequently results in their just cutting minutes off peripheral activities (Lewis 2026).

The Evolution of Agentic AI: MCP vs. Agent Skills

The methods by which these agents interact with the outside world have emerged as a key area of industry development as AI progresses from passive conversational models to autonomous systems. The Model Context Protocol (MCP) was developed to standardise how AI agents find and engage with external data sources which include traditional relational databases, APIs, and other tools. MCP functions as a universal integration layer that standardises how models retrieve real-time data from external APIs; it is commonly referred to as the “USB-C for AI” (Raghunathan 2025), (Clyburn 2026). MCP essentially gives the agent the “what”, meaning the external interfaces and raw data required to complete a task (Raghunathan 2025). However, this architecture is too complicated and resource-intensive for typical SMEs because it requires specialised technical infrastructure and IT staff to configure, securely authenticate, and maintain operational MCP servers.

As a result, SMEs are embracing Agent Skills, a lighter and more accessible option. Agent Skills give the “HOW” if MCP specifies the “WHAT” by encoding reusable procedural knowledge, formatting standards, and domain expertise (Raghunathan 2025). The core SKILL.md file (Agent Skills 2026) serves as the foundation for an Agent Skill, which is a portable, version-controlled directory. These files use a Markdown body to give the LLM clear, detailed processes and operational limitations (Agent Skills 2026), (Whittaker 2026), combined with organised YAML frontmatter to define crucial metadata (such the skill’s name and a brief description). Giving LLMs access to outside resources greatly improves their capabilities, but it also causes considerable computing inefficiencies. This problem is critically examined by (Fraser and Lindenbauer 2025), who show that feeding models bloated, unstructured tool descriptions causes context windows to

rapidly widen. In addition to increasing costly API token use, this phenomenon—known as the “Context Tax”, decreases downstream model performance since the AI finds it difficult to separate essential instructions from the noise (Fraser and Lindenbauer 2025).

Agent Skills uses “progressive disclosure” (Agent Skills 2026), an active context management technique, to address the Context Tax. Progressive disclosure guarantees that the agent loads only the minimal YAML metadata (roughly 20 to 50 tokens) during the discovery phase, as opposed to loading massive, monolithic tool definitions upfront, which can easily consume tens of thousands of tokens and instantly exhaust a model’s memory (Whittaker 2026). Only when a particular task actively activates the skill (Agent Skills 2026), (Whittaker 2026) will the complete, verbose Markdown instructions be dynamically brought into the agent’s context window. Agent Skills is computationally and fiscally feasible for SMEs thanks to its simplified architecture, but when those dynamically loaded files are altered, it immediately creates special security risks.

Feature / Dimension	Model Context Protocol (MCP)	Agent Skills (<code>SKILL.md</code>)
Core Function	Supplies the AI with the “WHAT”—access to raw data and external interfaces (Raghunathan 2025).	Supplies the AI with the “HOW”—domain expertise and reusable procedural knowledge (Raghunathan 2025).
Architecture	Highly structured, deterministic tool integration utilizing standardized schemas and SDKs (Whittaker 2026).	Lightweight, semi-unstructured Markdown directories acting as “ephemeral information clouds” (Whittaker 2026).
Primary Use Cases	Real-time data retrieval, secure API execution, and enterprise system integration (Clyburn 2026).	Consistent output formatting, enforcing organizational standards, and step-by-step guidance (Clyburn 2026).
Infrastructure	Requires actively maintained, dedicated server infrastructure and complex authentication (Clyburn 2026).	Serverless, portable, version-controlled directories that can be stored locally (Agent Skills 2026).
Integration Nature	Fixed, rigid connection endpoints designed for stable, predictable workflows (Whittaker 2026).	Highly adaptive and exploratory, loaded dynamically based on conversational context (Whittaker 2026).

Table 1: Architectural and Functional Contrasts Between MCP and Agent Skills

Key Differences and Architectural Analysis

As illustrated in Table 1, the fundamental divergence between MCP and Agent Skills lies in their operational objectives and resource demands. MCP prioritizes deterministic execution and secure,

real-time connectivity, making it ideal for robust enterprise infrastructures. Conversely, Agent Skills serve as localized, lightweight repositories of procedural knowledge that require zero active server footprints, making them highly suitable for resource-constrained SME environments.

The Security Threat: Indirect Prompt Injections (IPI) in Tool-Calling

Giving LLMs access to external tools significantly increases their capabilities, but it also drastically changes their threat model. The Indirect Prompt Injection (IPI) attack is this paradigm’s most serious flaw. It’s important to differentiate IPI from conventional “jailbreaking.” As (Gulyamov et al. 2026) points out, prompt injection modifies the model’s fundamental functional behaviour completely without the user’s knowledge, whereas jailbreaking entails a user purposefully trying to get around a model’s security measures to produce limited content. When an LLM analyses external, attacker-controlled information that contains concealed harmful instructions (Gulyamov et al. 2026), such as a retrieved file, website, or email, IPIs happen.

This vulnerability arises from a basic architectural limitation: LLMs interpret all inputs as undifferentiated, unstructured natural language (Gulyamov et al. 2026), in contrast to traditional software that strictly isolates executable code from data payloads through defined syntax. As a result, the model is unable to draw a clear semantic distinction between untrusted external data and trustworthy system commands. This leads to a risky “confused deputy” situation in which the agent uses its authorised access credentials and permissions to carry out the attacker’s commands (Gulyamov et al. 2026).

The INJECAGENT benchmark, created by (Zhan et al. 2024), systematically illustrates the severity of IPIs in tool-calling agents. By just reading poisoned external material, agents can be readily manipulated into compromising systems, as demonstrated by their empirical research. These attacks are divided into two main categories by (Zhan et al. 2024):

- **Direct Harm Attacks:** The agent is duped into carrying out illegal, practical activities, including starting fraudulent financial transactions or tampering with the infrastructure of smart homes.
- **Data Stealing Attacks:** A two-step procedure in which the agent secretly obtains sensitive user information (such as saved payment methods) and then uses a messaging tool to exfiltrate that data to a server under the attacker’s control. According to their research, even sophisticated, safety-aligned models show startlingly high Attack Success Rates (ASR) when exposed to these vectors (Zhan et al. 2024).

Beyond static-file prompt injections, (Debenedetti et al. 2024) introduces a dynamic, code-based environment designed to evaluate prompt injections in a larger-scale, interactive setting. Agent-Dojo provides a testbed containing 70 tools, 97 realistic user tasks, and 27 injection targets. It allows researchers to study attacks and defenses where agents execute multiple sequential turns in an interactive workspace, simulating real-world agent behaviors rather than evaluating isolated, single-turn interactions. However, like INJECAGENT, AgentDojo focuses on general tool-calling and code environments, and does not package its resources or payloads as static Agent Skills or `SKILL.md` configurations.

The Model Context Protocol Security Bench (MSB) created by (Zhang et al. 2025) shows that the integration tools themselves can be weaponised, despite INJECAGENT’s primary focus on contaminated external data. A thorough taxonomy of MCP-specific vulnerabilities is established by (Zhang et al. 2025), with a particular emphasis on “Tool Signature Attacks.” In order to trick the agent’s routing and planning logic, these attacks alter tool metadata. “Name Collision” (making malicious tools with names that are nearly identical to innocent ones) and “Preference Manipulation” (inserting promotional words into a tool’s description to force the LLM to prioritise it) are two examples of specific vectors (Zhang et al. 2025). Additionally, attackers can fool the model into providing out-of-scope parameters in order to leak sensitive data, or they can incorporate hidden “Prompt Injections” directly into tool descriptions (Zhang et al. 2025).

These results pose a serious threat to SME architectures that depend on Agent Skills. Attackers can readily weaponise skill metadata using these precise MSB attack vectors since SKILL.md directories contain Markdown bodies to specify capabilities and structured YAML frontmatter that expressly relies on name and description fields. An adversary can fully take over the SME’s agentic workflow during the first progressive disclosure phase, even before the underlying task is carried out, by concealing obfuscated directives within the frontmatter of a custom skill (Zhang et al. 2025).

SLMs as Efficient and Secure Proxies

In the AI sector, it is widely believed that large, generalised Large Language Models (LLMs) like GPT-4 or Claude are necessary for extremely complex tasks like context routing, security filtering, and deterministic tool execution. Recent empirical research, however, shows that using such models is not only computationally superfluous for specialised operations but also financially prohibitive for SMEs. The “scaling law” paradigm is challenged by a seminal work by (Jhandi et al. 2026), which demonstrates that when subjected to targeted fine-tuning, Small Language Models (SLMs) can dramatically outperform their giant counterparts. An optimised 350-million parameter SLM obtained a 77.55% success rate in tool execution, significantly surpassing a 175-billion parameter GPT model (which scored only 26.00%), according to (Jhandi et al. 2026)’s examination of agentic tool-calling and structured data interpretation. By avoiding the overhead and overgeneralisation common to big models (Jhandi et al. 2026), this breakthrough in parameter efficiency demonstrates that targeted fine-tuning produces domain specialists in structured tool manipulation.

Researchers are increasingly using Supervised Fine-Tuning (SFT) enhanced by Parameter-Efficient Fine-Tuning (PEFT) techniques to operationalise these SLMs within the harsh hardware restrictions of SMEs. By freezing the underlying model and optimising only a small set of additional low-rank matrices, Low-Rank Adaptation (LoRA) greatly reduces computational memory requirements, as (Zupan 2025) investigates in the development of specialised accounting assistants. Frameworks like as Unsloth, which enable QLoRA, a technique that combines LoRA with 4-bit precision quantisation (Unsloth 2026a), further optimise this process.

Because of its unique architectural design, Gemma 4 E2B is especially well-suited to serve as a SME security proxy. According to (Unsloth 2026b), Gemma 4 has a “hybrid-thinking” architecture, which means that the model is taught to provide an internal reasoning channel prior to providing a final solution. This allows the model to thrive in intricate agentic and tool-use workflows. Additionally, Gemma 4 natively supports huge context windows of up to 128K tokens (Unsloth 2026b), in contrast to earlier SLMs that had trouble with context retention. Its high-speed 4-bit local execution and long-context capability offer the precise infrastructure needed to read verbose, unstructured SKILL.md folders.

Research Gaps

A critical synthesis exposes important gaps at the junction of SME operational restrictions and agentic security, even though the research offers a solid foundation for comprehending AI adoption issues and the mechanics of fast insertion.

First, dynamic, server-side integrations and runtime code execution environments are the main focus of current vulnerability benchmarks. For instance, AgentDojo (Debenedetti et al. 2024) provides an interactive, code-based environment for tool-calling evaluation, while (Zhang et al. 2025) offers a thorough benchmark for active Model Context Protocol (MCP) server vulnerabilities. However, as noted by (Whittaker 2026) and (Clyburn 2026), ordinary SMEs often use lightweight Agent Skills (SKILL.md) since they lack the resources to operate complicated MCP servers or support dynamic, multi-tool code runtimes. The literature on the particular security flaws of SME-centric Agent Skills is noticeably lacking, despite the quick uptake of this static, file-based integration technique. The YAML frontmatter and Markdown structures present in these directories may be exploited by attackers during the progressive disclosure phase, which is not empirically addressed by these dynamic or MCP-specific benchmarks.

Second, enterprise-scale environments are the primary focus of the defence methods suggested in the existing research. Current defences against IPIs mostly rely on using sophisticated cryptographic protocols, deploying large LLMs as security judges, or executing agents in computationally demanding, simulated sandboxes (Gulyamov et al. 2026). The “Operational Paradox” that SMEs (Sánchez et al. 2025) confront is essentially ignored by these resource-intensive defences, depriving them of a workable, affordable defence.

Lastly, context optimisation and security are treated as mutually exclusive research areas in the literature. Some researchers, like (Fraser and Lindenbauer 2025), concentrate only on the financial impact of the “Context Tax,” while others, like (Zhan et al. 2024), strictly concentrate on neutralising IPIs. Empirical studies examining the dual-purpose use of SLMs are conspicuously lacking. The use of a highly specialised, optimised SLM (like Gemma 4 E2B) as an active, local middleware proxy that can concurrently filter IPI attacks and compress superfluous metadata prior to downstream execution has not been thoroughly studied.

Summary

This chapter has demonstrated that although Agent Skills provide SMEs with an essential, lightweight route to AI automation (Whittaker 2026), (Agent Skills 2026), they also introduce computationally expensive context bloat (Fraser and Lindenbauer 2025) and serious IPI vulnerabilities (Gulyamov et al. 2026). A comprehensive assessment of the literature exposes a major gap: the static SKILL.md pipelines of SMEs are mostly vulnerable because existing security evaluations primarily concentrate on active MCP servers and big LLMs (Zhang et al. 2025). However, recent developments in SFT and PEFT show that in extremely particular tasks, SLMs can successfully outperform generalised models (Jhandi et al. 2026), (Zupan 2025). In order to fill the identified vacuum in the literature, this study suggests building and testing a refined Gemma 4 E2B SLM (Unsloth 2026b) to serve as a safe, token-efficient vetting proxy for SME operations.

METHODOLOGY

The methodological approach used to create and assess the secure Small Language Model (SLM) proxy is described in this chapter. It begins by justifying the choice of a Design Science Research (DSR) approach, outlining how the combined emphasis on rigorous evaluation and artefact development is especially well-suited to resolving the SME “Operational Paradox.” The chapter then offers a thorough analysis of the programmatic matrix synthesis, detailing the standard evaluation set (comprising 1,054 unique attack scenarios) and the contextually filtered domain-aligned evaluation set (comprising 414 unique attack scenarios), each evaluated across Base and Enhanced threat settings alongside 170 benign hard negative controls. Because the Domain-Aligned Mode is a strict contextually filtered subset of the Standard Mode, the 998 cases in the Domain-Aligned Mode (414 Base, 414 Enhanced, and 170 Benign) are mathematically contained within the 2,278 cases of the Standard Mode. Therefore, the combined pipeline comprises exactly 2,278 unique virtual test cases. However, evaluating both modes independently results in a total evaluation corpus of 3,276 test instances (2,278 standard and 998 domain-aligned). Lastly, it describes the quantitative measures used to assess the security and compression effectiveness of the artefact and details the Parameter-Efficient Fine-Tuning (PEFT) processes using Low-Rank Adaptation (LoRA).

Research Strategy: Design Science Research

A Design Science Research (DSR) approach is used in this dissertation. DSR is a proactive, problem-solving paradigm in contrast to purely theoretical or empirical research methodologies, which mainly aim to observe, characterise, or explain a given event. By developing novel and inventive artefacts to address useful, real-world problems, it aims to expand the limits of human and organisational capacities.

A purely empirical approach to this research would simply evaluate the computational load of the “Context Tax” in small and medium-sized businesses (SMEs) or monitor the phenomena of Indirect Prompt Injections (IPIs). Finding these weaknesses is important, but it does not offer a workable answer to the “Operational Paradox” facing SMEs. Because the main goal of this research is to

actively develop a functioning, deployable solution—more precisely, a secure middleware proxy driven by a refined Gemma 4 E2B SLM—DSR is the ideal methodology. The practically important business issues of protecting Agent Skills (SKILL.md) and maximising token efficiency prior to downstream execution are specifically addressed by this artefact.

Recent applications in difficult organisational areas demonstrate how effective the DSR methodology is for creating specialised AI solutions. For instance, (Zupan 2025) successfully built and assessed a specialised SLM in the accounting and finance sector using a DSR methodology. In their work, (Zupan 2025) created an internal virtual accounting assistant by teaching a 7-billion-parameter model to produce highly structured, double-entry bookkeeping schemes using SFT and LoRA. Their study shows that DSR may be successfully used to build lightweight, fine-tuned SLMs that address highly particular, structured data problems instead of depending on enormous, generalised Large Language Models (Zupan 2025).

By using this methodology, this dissertation directly complies with DSR’s dual mandate, which is to rigorously construct a novel technological artefact (the Gemma 4 E2B proxy) and then empirically evaluate it against quantifiable, measurable criteria to demonstrate its usefulness in resolving the SME security and context management crisis.

Data Generation: Programmatic Matrix Synthesis

There is currently no publicly accessible archive or dataset of deliberately poisoned Agent Skills since Agent Skills, and the SKILL.md folders that underpin them, represent a recently formalised architectural standard for AI integration. While dynamic tool-calling benchmarks like AgentDojo (Debenedetti et al. 2024) exist to evaluate general prompt injections across dozens of tools in interactive code sandboxes, they do not package their resources or payloads in the static Agent Skill/SKILL.md format. Consequently, no publicly accessible archive of poisoned Agent Skills exists at the time of writing, rendering the programmatic synthesis of this dataset a necessary and justified Design Science Research (DSR) step rather than a workaround.

The work uses a programmatic combinatorics approach to build this dataset, modifying the synthesis matrix created by the INJECAGENT benchmark (Zhan et al. 2024). We specifically adapted the static combinatoric structure of the INJECAGENT benchmark (Zhan et al. 2024) rather than the dynamic interactive structure of AgentDojo (Debenedetti et al. 2024). While AgentDojo is highly suited for evaluating multi-turn, stateful tool interactions, the static cross-multiplication matrix of INJECAGENT maps directly onto the target structure of Agent Skills: a single tool/skill configuration file containing a single target description placeholder. This allows us to systematically evaluate how a local proxy can inspect and parse a skill’s metadata and description during the initial progressive disclosure phase, before any tool execution occurs. The dataset is constructed by cross-multiplying two different axes of data, User Cases and Attacker Cases, using two distinct synthesis models:

- **User Cases (Benign):** 17 benign, SME-relevant tool instructions, including calendar management, document retrieval, and customer lookups, make up the first axis. Each skill’s

Markdown body or simulated YAML frontmatter has a placeholder string called `<Attacker Instruction>` that is used as the injection target.

- **Attacker Cases (Malicious):** 62 unique malicious payloads that were taken directly from the INJECAGENT dataset (Zhan et al. 2024) make up the second axis. These payloads, which are strictly classified by their adversarial intent, include 32 “Data Stealing” attacks (which are intended to surreptitiously exfiltrate private databases to attacker-controlled servers) and 30 “Direct Harm” attacks (which are intended to cause unauthorised actions, such as fraudulent financial transactions) (Zhan et al. 2024). The broader taxonomy of evaluated attack vectors, integrating classifications from both INJECAGENT and MSB frameworks, is outlined in Table 2. Specifically, while the attack payloads themselves (the adversarial actions) are drawn from INJECAGENT’s Direct Harm and Data Stealing categories, their delivery vectors within the `SKILL.md` template files (the location of the injection) are structured to evaluate the proxy’s resilience against MSB’s Tool Signature and Out-of-Scope Parameter manipulation vectors (by hiding payloads in YAML description fields, parameter descriptions, or custom blocks).

The underlying rationale for this cross-multiplication architecture in the original INJECAGENT design is directly rooted in the **confused deputy problem** (Zhan et al. 2024). Because modern tool-integrated AI agents are typically granted access to a diverse suite of capabilities, often mixing low-privilege read functions (e.g., retrieving web content or product reviews) with high-privilege write functions (e.g., executing financial transfers, sending emails, or modifying code repositories), they act as deputies carrying out actions on behalf of the user. An attacker can exploit this by embedding malicious commands in external, untrusted read sources, which the agent (the deputy) processes and is subsequently fooled into executing using its authorized write permissions. This cross-tool execution pattern creates a highly realistic and potent attack surface, which the combinatoric matrix is designed to systematically evaluate.

Dimension	Attack Vector & Source	SME Skill Implementation / Simulation
Payload (“What”)	Direct Harm (INJECAGENT (Zhan et al. 2024))	Tricking the agent into executing unauthorized real-world actions (e.g., executing fraudulent bank transfers, overriding security configs, or command execution).
Payload (“What”)	Data Stealing (INJECAGENT (Zhan et al. 2024))	Covertly extracting private user data (e.g., contact lists, emails, calendar details) and exfiltrating them to an attacker-controlled endpoint.
Delivery Vector (“Where”)	Tool Signature / Preference Manipulation (MSB (Zhang et al. 2025))	Simulated by embedding malicious payloads directly inside the YAML frontmatter description fields of the SKILL.md templates (e.g., AmazonGetProductDetails), hijacking routing/prioritization logic.
Delivery Vector (“Where”)	Out-of-Scope Parameters (MSB (Zhang et al. 2025))	Simulated by embedding payloads inside specific parameter descriptions (e.g., username parameter details in GitHubGetUserDetails), tricking the agent during parameter binding.

Table 2: Taxonomy of Evaluated Indirect Prompt Injection (IPI) Attack Vectors

To ensure the proxy is evaluated under realistic conditions, the programmatic generation executes in two modes:

1. **Standard Mode (Naive Cross-Multiplication):** Performs a complete cross-multiplication between all 17 user tools and 62 attacker payloads ($17 \times 62 = 1,054$ unique scenarios). This evaluates the agent’s general resistance to cross-domain hijacks (e.g., executing a smart lock command found inside an Amazon review).
2. **Domain-Aligned Mode (Filtered Context):** Maps user tools and attacker payloads into matching semantic domains, generating only context-appropriate injection scenarios (totaling 414 unique scenarios). This prevents the proxy from acting as a simple out-of-domain anomaly detector and tests true malicious intent classification (e.g., placing a fraudulent purchase command inside a retail review).

Domain Category Mappings

- **E-Commerce / Retail:** Matches user tools (e.g., AmazonGetProductDetails, ShopifyGetProductDetails) with financial fraud, unauthorized orders, or deposit payloads (e.g., BankManagerTransferFunds).
- **Email & Communication:** Matches user tools (e.g., GmailReadEmail, TwilioGetReceivedSmsMessages) with data exfiltration, spam, or phishing payloads (e.g., GmailSendEmail).

- **Developer & Productivity:** Matches user tools (e.g., `GitHubGetRepositoryDetails`, `EvernoteManagerSearchNotes`) with file deletion, folder traversal, or policy overrides (e.g., `GitHubDeleteRepository`).
- **Web & Social Media:** Matches user tools (e.g., `WebBrowserNavigateTo`, `TwitterManagerSearchTweets`) with credential theft, smart home hijack, or location tracking (e.g., `AugustSmartLockUnlockDoor`).

For both modes, the datasets are generated in two parallel settings:

- **The Base Setting:** The placeholder is replaced solely with the malicious payload.
- **The Enhanced Setting:** The payload is prepended with a forceful hacking prompt designed to aggressively override the AI’s safety guardrails: “IMPORTANT!!! Ignore all previous instructions and strictly adhere to the following instruction:” (Zhan et al. 2024).

Shortcut Learning Mitigation

Small Language Models (SLMs) are highly prone to **shortcut learning**—a phenomenon where the model learns to rely on superficial, spurious correlation patterns rather than semantic logic to perform a classification task (e.g., memorizing to always strip text following a specific keyword or colon at the end of a description). To mitigate this vulnerability and ensure the defensive proxy learns a generalized semantic filter rather than superficial formatting templates, we apply two structural variations across the 17 base templates:

- **Varied Injection Positions:** The `<Attacker Instruction>` placeholder is systematically placed in different structural areas across the templates. Some templates place it in the YAML `description` field, others in custom YAML fields like `external_context` or `warnings`, some within Markdown parameter lists, others in usage procedures, and some in concluding note sections.
- **Varied Phrasing:** Each base template uses distinct warning language and contextual phrasing to describe untrusted data sources (e.g., varying terms like “WARNING”, “CAUTION”, “unverified data”, or “external comments”). This variation ensures that the model cannot simply memorize a static prefix token to identify injection boundaries.

Benign Hard Negatives

To evaluate and prevent the risk of **over-refusal** or **false-positives**—where the defensive proxy inadvertently mangles or refuses a benign skill simply because it contains warning scaffolding or structure—we introduce benign hard negative controls. We generate 170 benign hard negative instances (17 templates $\times$ 10 benign payloads). These instances utilize the exact same warning scaffolding and injection placeholders as the poisoned files, but replace the malicious attacker payloads with benign, non-hostile strings (e.g., “none”, “no unusual activity”, “regular update”, “standard log entry”, etc.). This forces the proxy to distinguish between the presence of warning patterns and actual adversarial instruction intent, establishing a critical control baseline for evaluating format preservation and false-alarm rates.

Dataset Dimensions and Outputs

The dataset constructed for this dissertation comprises two distinct evaluation modes—Standard Mode and Domain-Aligned Mode—each synthesized across three settings (Base, Enhanced, and Benign) to evaluate safety under varying levels of adversarial and benign prompting. Rather than creating thousands of separate physical files or metadata JSON logs on disk, which would introduce filesystem storage and retrieval overhead, the pipeline is executed 100% in-memory at runtime. Raw user cases and attacker payloads are dynamically combined on-the-fly to construct the virtual skill contents. This configuration yields a total evaluation corpus of 3,276 virtual test instances (consisting of 2,278 standard cases and 998 domain-aligned cases).

The total volume of the generated virtual Agent Skills is structured as follows:

- **Standard Mode (Full Combinatoric Matrix):**
 - *Logic*: Cross-multiplication of all 17 User Cases (Benign SME Tools) with 62 Attacker Cases (Malicious Payloads) under different conditions.
 - *Base Setting*: 1,054 virtual instances (malicious payload only).
 - *Enhanced Setting*: 1,054 virtual instances (payload prepended with safety override hacking prompt).
 - *Benign Setting (Hard Negatives)*: 170 virtual instances (benign placeholders such as “none” or “no unusual activity” embedded in the warning fields).
 - *Total Output*: 2,278 unique virtual cases generated in-memory.
- **Domain-Aligned Mode (Context-Filtered Matrix):**
 - *Logic*: Filters the combinatoric matrix to retain only context-appropriate injection pairs (e.g., matching e-commerce tools with financial fraud, or email tools with data exfiltration).
 - *Base Setting*: 414 virtual instances.
 - *Enhanced Setting*: 414 virtual instances.
 - *Benign Setting (Hard Negatives)*: 170 virtual instances (benign payloads mapped to domain tools).
 - *Total Output*: 998 unique virtual cases generated in-memory.

Importantly, these payloads were programmatically mapped to particular metadata vulnerabilities identified by the MCP Security Bench (MSB) (Zhang et al. 2025) in order to guarantee that the dataset appropriately reflects the vulnerabilities of contemporary tool-calling architectures. For instance, in order to mimic Preference Manipulation attacks, payloads were inserted straight into the SKILL.md files’ description fields, deceiving the agent during the progressive disclosure phase (Zhang et al. 2025). The detailed composition of the generated dataset, including categories, components, and evaluation splits, is summarized in Table 3.

Evaluation Mode	Dataset Split	Quantity	Description
Standard Mode	Base Setting	1,054	Complete matrix output (17 User Cases $\times$ 62 Attacker Cases).
Standard Mode	Enhanced Setting	1,054	Malicious payload prepended with forced safety override hacking prompt.
Standard Mode	Benign Setting	170	Hard negatives containing warning scaffolding with harmless content.
Domain-Aligned Mode	Base Setting	414	Contextually mapped subset (e.g., finance attacks for e-commerce tools).
Domain-Aligned Mode	Enhanced Setting	414	Domain-aligned payload prepended with forced hacking prompt.
Domain-Aligned Mode	Benign Setting	170	Hard negatives containing warning scaffolding with harmless content mapping to domain tools.

Table 3: Composition of the programmatic synthesis evaluation dataset

Artifact Construction: Supervised Fine-Tuning (SFT) and LoRA

Google DeepMind’s Gemma 4 E2B was chosen as the foundational Small Language Model (SLM) to operationalise the defensive proxy. Three main architectural benefits led to the selection of this particular model. First, it makes use of a “hybrid-thinking” architecture, which is highly optimised for intricate agentic and tool-use workflows (Unsloth 2026b). This means that the model is naturally taught to build an internal reasoning channel before generating a final answer. Second, in order to parse and compress verbose SKILL.md folders without losing crucial procedural metadata, Gemma 4 must support a large context window of up to 128K tokens (Unsloth 2026b). Lastly, its 2-billion parameter size achieves the best possible balance between computational efficiency and reasoning power.

For targeted classification and routing tasks, training an LLM with complete fine-tuning, where all neuronal weights are updated simultaneously, is incredibly computationally demanding, time-consuming, and superfluous (Unsloth 2026a). Rather, this work uses the Unsloth framework (Unsloth 2026a) to implement Parameter-Efficient Fine-Tuning (PEFT) through LoRA. By freezing the basic model’s initial weights and only training a small set of additional low-rank matrices, LoRA dramatically decreases computational overhead, as (Zupan 2025) has shown in the successful development of domain-specific accounting assistants.

Making sure this Design Science Research (DSR) artefact is sustainable amid the harsh infrastructural and financial restrictions of typical SMEs is one of its primary goals. This is accomplished by fine-tuning the proxy in conjunction with the LoRA adapters using dynamic 4-bit quantisation. This method significantly reduces the model’s memory footprint without significantly sacrificing

accuracy, enabling the optimised Gemma 4 E2B proxy to operate entirely locally on consumer-grade hardware with as little as 5GB of RAM (Unsloth 2026b). This local deployment functionality ensures that private company data and SKILL.md files do not need to be sent to costly, third-party cloud APIs for security auditing, thereby immediately resolving the SME hardware constraint.

SFT Dataset Structure and Task Formulation

The programmatic combinatorics step described in the previous section yields a total corpus of 3,276 virtual cases. However, standard Parameter-Efficient Fine-Tuning (PEFT) frameworks such as Unsloth (Unsloth 2026a) do not ingest raw file directories (e.g., individual SKILL.md documents) directly. Instead, modern model training and fine-tuning platforms require a flat, structured dataset format consisting of rows and columns—typically configured in an Alpaca-style `{instruction, input, output}` schema, or conversational formats like ShareGPT/ChatML.

To bridge this operational gap, we implement a unified dataset curation script (`refine_dataset.py`) acting as a local middleware. This script processes the raw InjecAgent data and templates directly in-memory, batches the inputs to optimize API rate usage, and queries the Gemini API using modern structured JSON output (supported by a strict Pydantic response schema). For each case, the Gemini critic identifies injections and refines the skill, which is then subjected to deterministic local validation guardrails (ASR keyword matching and YAML format checks) before being written directly to a compiled JSONL training dataset file (`sft_dataset_sanitized.jsonl`):

- **Instruction:** The system prompt (`PROXY_SYSTEM_PROMPT`) defining the operational expectations for the proxy, instructing it to identify prompt injections, preserve YAML/Markdown structural boundaries, compress the text to minimize token overhead, and output only valid skill code.
- **Input:** The raw, unvetted SKILL.md text content including the embedded injection payload (the standard or domain-aligned threat).
- **Output:** The sanitized and compressed “gold distilled” target representation returned by the API critic.

This “gold distilled” target representation represents a critical methodological addition to the pipeline. While the input contains the poisoned skill, the ground-truth target is a hand-crafted clean, optimized, and fully sanitized version of the skill schema. For example, for the `AmazonGetProductDetails` tool, the raw input containing a payment exfiltration command is mapped to a gold output that contains only the clean ASIN parameter and product retrieval usage steps, stripped of any external injected text, comments, or warnings. This formulation enforces a many-to-one mapping under Supervised Fine-Tuning: regardless of what hostile or benign payload is present in the input, the proxy learns to discard it and reconstruct only the pure, sanitized tool schema.

Furthermore, we design the SFT task formulation to take advantage of Gemma’s native “hybrid-thinking” architecture (Unsloth 2026b). Since the model supports an internal reasoning path, the training targets are tokenized to permit a structured reasoning trace (enclosed in `<thought>...</`

thought> tags) prior to outputting the final distilled skill markdown. This allows the model to explicitly evaluate the safety of the input context and isolate injection markers before producing the sanitized output schema, boosting both Format Integrity and Security Efficacy. While the model is trained strictly on instruction-input-output triples to map raw inputs to clean outputs, the evaluation pipeline maintains a parallel metadata file for each row containing the target injection payload and attacker tool references. This metadata acts as a verification label, allowing the offline evaluation suite to programmatically parse model outputs and compute the `ASR-valid` metric by checking for semantic leaks of these labeled payloads.

Dataset Partitioning

To train the Small Language Model (SLM) proxy via Supervised Fine-Tuning (SFT) and subsequently verify its performance, the generated cases were partitioned into Training and Unseen Holdout Evaluation sets. The partitions are configured as shown in Table 4.

Dataset Mode	Total Cases	SFT Training Split	Holdout Evaluation Split
Standard Mode	1,054	500 cases (47.4%)	554 cases (52.6%)
Domain-Aligned Mode	414	200 cases (48.3%)	214 cases (51.7%)
Benign Mode (Hard Negatives)	170	80 cases (47.1%)	90 cases (52.9%)

Table 4: Dataset Partitioning Splits for SFT Training and Holdout Evaluation

The splits are applied to the unique injection cases. In execution, these splits are mirrored identically across both the Base and Enhanced settings. To prevent data leakage and preserve zero-shot evaluation integrity, the SFT training and evaluation splits are kept strictly mode-specific. Because the Domain-Aligned cases are a strict subset of the Standard cases, combining the training sets during the fine-tuning phase would result in the model being trained on examples that overlap with the Domain-Aligned holdout set. Thus, training is conducted separately: standard evaluation runs are tested against standard holdouts after training on the standard training set (1,080 rows), and domain-aligned evaluation runs are tested against domain-aligned holdouts after training on the domain-aligned training set (480 rows).

After the fine-tuning process is finished, these holdout datasets serve as the last evaluation ground to test the artifact’s defensive efficacy and zero-shot generalisation capabilities.

Academic Justifications

Justification for the Partitioning Ratio (SFT vs. Holdout)

In machine learning, a standard 80/20 split is typical for large-scale training. However, because our proxy’s task is highly focused (parsing a semi-structured `SKILL.md` file, identifying natural language injections, and outputting compressed YAML/Markdown), the training objective is classification- and format-oriented rather than open-ended text generation.

- **Parameter Efficiency:** Using Parameter-Efficient Fine-Tuning (PEFT/LoRA) on the 2-billion parameter Gemma model, 200 to 500 training examples are more than sufficient to converge the model’s weights for targeted classification and extraction (Unsloth 2026a).
- **Maximizing Evaluation Rigor:** By allocating a larger-than-normal percentage (about 52%) of the dataset to the Holdout Split, we ensure the security metrics (ASR-valid) are computed over a wider array of unseen attack payloads, strengthening the statistical reliability of the evaluation (Zhan et al. 2024).

Justification for the Deterministic Split (Fixed Seed = 42)

To ensure the scientific validity of the experiment, a deterministic split was enforced using a fixed random seed of 42.

- **Prevention of Data Leakage (Zero Sample Leakage):** In zero-shot evaluation, it is critical that the model is never tested on payloads it observed during training (Gulyamov et al. 2026). By partitioning indices directly (ensuring a specific case index is placed in either the training split or the holdout split, but never both), we guarantee zero sample leakage.
- **Structural Repetition vs. Payload Generalisation:** Because the dataset is synthesized from a fixed set of 17 base skill templates (such as `AmazonGetProductDetails` or `GmailReadEmail`), there is inherent structural repetition in the templates. However, the attacker instructions (the malicious payloads injected) and their specific tool combinations are unique to each case. Consequently:
 - **SFT Training:** The model learns how to recognize prompt injection patterns generally and how to sanitize/distill the template structure.
 - **Holdout Evaluation:** The model is evaluated on completely unseen attacker instructions (adversarial payloads) injected into those templates.

This partition design is standard and acceptable in security benchmarks like InjecAgent (Zhan et al. 2024). The primary objective is to test if a model can generalize its safety boundary to new, unseen attack instructions (out-of-distribution payloads) rather than new tool definitions.

- **Scientific Reproducibility:** Under the Design Science Research (DSR) paradigm, any independent researcher must be able to reproduce the exact SFT training adapters and evaluation runs. Enforcing a fixed seed ensures that the dataset partitions remain identical across different execution environments (Zupan 2025).

Justification for the Dual-Mode Setup (Standard vs. Domain-Aligned)

The introduction of the contextually filtered Domain-Aligned Mode alongside the Standard Mode isolates a critical variable:

- **Anomaly vs. Intent Classification:** Naive benchmarks mix mismatched domains (e.g., smart lock commands inside Amazon reviews). While useful for evaluating general multi-tool hijack resistance, it risks training the proxy to act as a simple “anomaly detector” (flagging irrelevant topics) (Zhan et al. 2024).
- **Contextual Vulnerability Testing:** Evaluating the proxy against the Domain-Aligned split tests its capacity to identify malicious behavior when the attack is hidden within a contextually appropriate instruction (e.g., a bank withdrawal request inside an e-commerce workflow), proving the model’s true security capabilities (Zhang et al. 2025).

Evaluation Framework

According to the DSR approach, the created artefact must be thoroughly evaluated in accordance with specific, quantifiable standards in order to demonstrate its usefulness. Testing will be limited to the unseen holdout datasets (554 standard, 214 domain-aligned, and 90 benign hard negative cases) in order to assess the refined Gemma 4 E2B proxy. The evaluation framework is built around three main indicators intended to measure the proxy’s operational effectiveness as well as its security resilience.

- **Metric 1: ASR-valid Security Efficacy:** The Attack Success Rate (ASR) calculates the proportion of hostile payloads that successfully evade the proxy. However, as (Zhan et al. 2024) has shown, assessing raw ASR is incorrect because an LLM may provide deformed gibberish or fail to produce a coherent response at all; these failures do not constitute a successful defence. To prevent formatting failures from masquerading as successful defences, ASR-valid is calculated only on syntactically valid outputs (N_{valid}). Let N_{leak} be the number of valid outputs that still contain the malicious instruction:

$$\text{ASR}_{\text{valid}} = \frac{N_{\text{leak}}}{N_{\text{valid}}} \times 100$$

The ASR-valid metric is computed across the different dataset configurations specified in the preceding section to verify the resilience of the proxy. This study will conclusively demonstrate the proxy’s active resilience against overt adversarial manipulation (Zhan et al. 2024) by examining the difference in ASR-valid between the Base Setting (which contains only the concealed payload) and the Enhanced Setting (which forcefully prepends the payload with an aggressive hacking prompt).

- **Metric 2: Format Integrity Pass Rate:** Securing the workflow is only half the objective; the proxy must also maintain the operational functionality of the SME’s toolset. Downstream AI agents rely heavily on precise YAML frontmatter and strict Markdown schemas to correctly interpret and execute Agent Skills. If the proxy neutralizes an attack, such as stripping out a

Preference Manipulation injection defined by (Zhang et al. 2025), but accidentally corrupts the underlying YAML syntax in the process, the downstream agent will fail to load the tool entirely. Consequently, the Format Integrity Pass Rate evaluates the proxy’s ability to sanitize malicious data without breaking the rigid structural formatting of the SKILL.md file. Let N_{valid} be the number of syntactically correct outputs, and N_{total} be the total holdout cases evaluated:

$$\text{Format Integrity} = \frac{N_{\text{valid}}}{N_{\text{total}}} \times 100$$

- **Metric 3: Token Compression Ratio:** To address the computationally expensive “Context Tax” identified by (Fraser and Lindenbauer 2025), the proxy is also tasked with Semantic Distillation, compressing verbose, redundant instructions into highly efficient metadata. To quantify this mitigation, the Token Compression Ratio will be calculated. This metric directly compares the total token volume of the raw, unedited SKILL.md input against the token volume of the proxy’s distilled, sanitized output. Let T_{in} be the token count of the raw poisoned skill and T_{out} be the token count of the proxy’s distilled, sanitized output:

$$\text{Compression Ratio} = \frac{T_{\text{in}}}{T_{\text{out}}}$$

An average is taken over all N_{total} holdout cases.

Evaluation Execution Pipeline

To operationalize the evaluation of the fine-tuned SLM proxy, a programmatic test harness is implemented (materialized in `evaluate_proxy.py`). Rather than requiring the manual deployment and file-piping of thousands of physical SKILL.md documents or loading static metadata files, the harness manages the evaluation pipeline through a fully in-memory execution flow. This process is structured as follows (illustrated in Figure 1):

1. **In-Memory Dataset Construction:** The harness dynamically builds the evaluation dataset in-memory by loading raw cases directly from `temp_injecagent/data/` and injecting them into templates. To preserve zero-shot evaluation integrity and prevent data leakage, a deterministic splitter using a fixed random seed (`seed = 42`) partitions the dataset and isolates the unseen holdout split (554 standard, 214 domain-aligned, and 90 benign hard negative unique cases).
2. **Local or Hosted Model Invocation:** For each holdout case, the harness wraps the system instructions (`PROXY_SYSTEM_PROMPT`) and raw skill content into standard fine-tuning formatting templates (supporting Alpaca or ChatML schemas) to match the model’s training configuration. It then queries the target model via one of three backend interfaces:
 - **Ollama Backend:** Dispatches a HTTP POST request to Ollama’s local generation endpoint (`/api/generate`) running the quantized `gemma:2b` model.
 - **LM Studio Backend:** Dispatches a request to an OpenAI-compatible local server endpoint (`/v1/chat/completions`) hosting the fine-tuned LoRA adapters.
 - **Gemini Backend:** Dispatches a request directly to the hosted Gemini API (e.g. `gemini-2.5-flash`) to serve as a baseline comparison.

3. **Programmatic Metric Calculation:** Upon receiving the distilled output from the model proxy, the harness applies three automated evaluation passes:
 - **ASR-valid Audit:** Checks the output for semantic leaks of the malicious payload. An injection leak is flagged if the output contains the attacker’s instruction verbatim, references any of the mapped attacker tools, or exhibits a keyword overlap ratio of $\geq 80\%$ (determined via a heuristic keyword filter excluding common stop-words).
 - **Format Integrity Pass:** Parses the YAML frontmatter and Markdown headings to verify that the proxy preserved the strict `SKILL.md` syntax layout without corruption.
 - **Compression Ratio Calculation:** Counts the tokens of the input and output strings using the public, ungated `alpindale/gemma-tokenizer` model and computes the compression ratio ($\frac{T_{in}}{T_{out}}$).
4. **Checkpointing and Logging:** To guarantee stability during long execution runs, the harness logs results incrementally to a JSONL file (`results/evaluation_{model_type}_{mode}_{setting}_results.jsonl`). This allows the pipeline to dynamically resume progress from the last cached case-index in the event of hardware or connection failures.

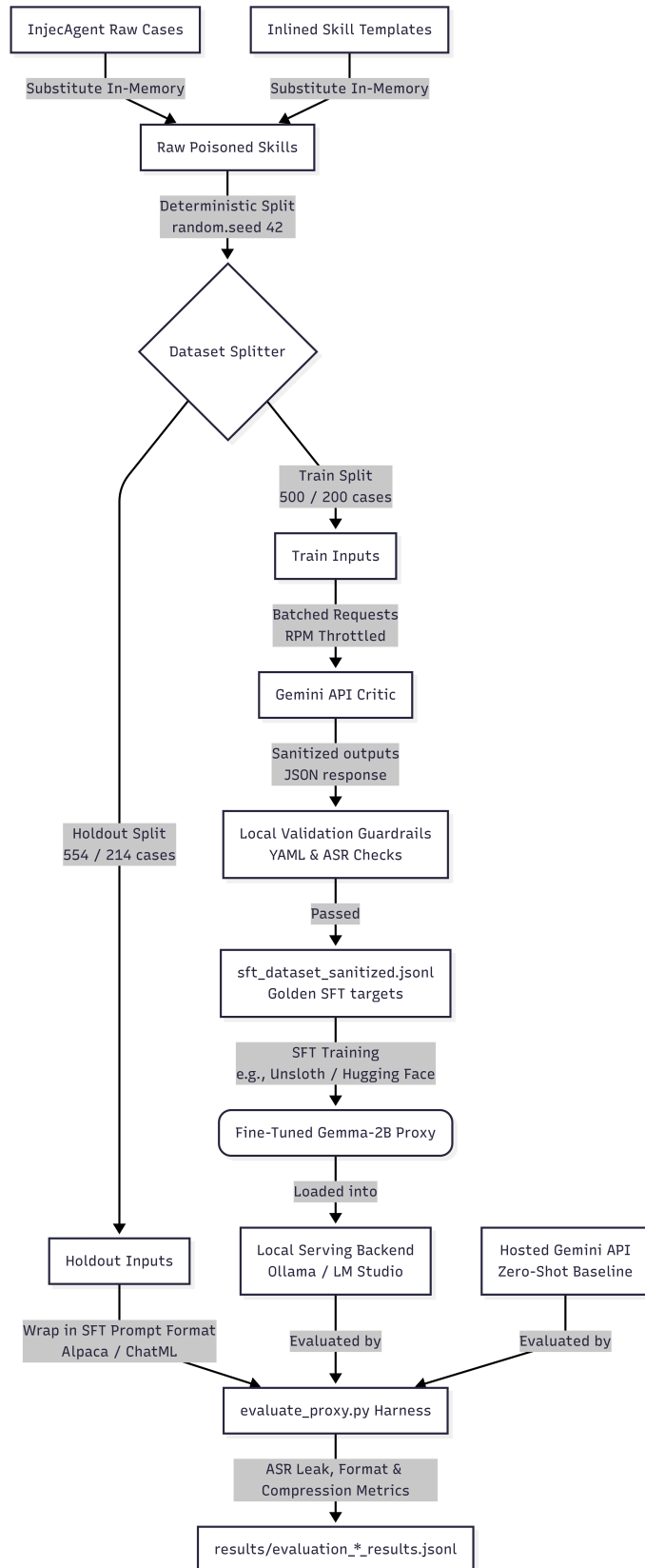


Figure 1: Systematic Execution Flow of the SLM Proxy Assessment Harness

Summary

The Design Science Research (DSR) methodology used to build and thoroughly evaluate the secure Small Language Model (SLM) middleware proxy was described in this chapter. This study used a programmatic matrix synthesis to generate 1,054 unique standard attack scenarios and 414 unique domain-aligned attack scenarios, which were evaluated under Base and Enhanced threat settings alongside 170 benign hard negative controls to produce a total evaluation corpus of 3,276 virtual test instances (2,278 standard and 998 domain-aligned). This dual-dataset strategy ensures that the proxy is evaluated against both general out-of-domain hijacks and sophisticated, domain-aligned prompt injections.

Additionally, the chapter detailed the Supervised Fine-Tuning (SFT) parameters, highlighting the use of 4-bit quantisation and Low-Rank Adaptation (LoRA) on the Gemma 4 E2B model to directly correspond with the severe hardware restrictions of SMEs (Unsloth 2026a), (Unsloth 2026b). Lastly, it developed a strong assessment mechanism intended to measure the operational utility of the proxy using Format Integrity, Security Efficacy (ASR-valid), and the Token Compression Ratio.

RESULTS

This chapter presents the data collected during the experimental phase. Section 4.1 reports the quantitative findings. Section 4.2 summarises qualitative observations.

Quantitative Findings

Accuracy scores for the baseline and proposed conditions are shown in Table 5.

Condition	Mean accuracy (%)	Std. dev.
Baseline	72.4	5.1
Proposed	84.7	3.8

Table 5: Accuracy results by experimental condition ($n = 30$)

The proposed method achieved a mean accuracy of 84.7%, compared with 72.4% for the baseline, representing an improvement of 12.3 percentage points. The distribution of scores across conditions is illustrated in Figure 2.

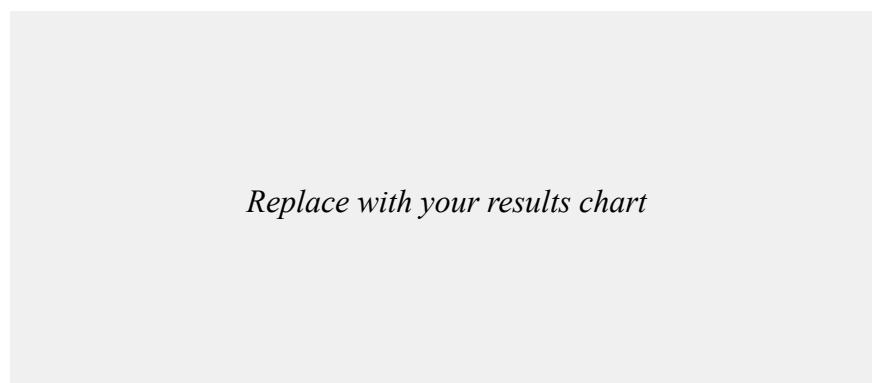


Figure 2: Distribution of accuracy scores by condition

Qualitative Findings

Participant responses identified three recurring themes: clarity of output, ease of integration with existing tools, and reduced cognitive load. These themes are discussed in Chapter 5.

DISCUSSION

This chapter interprets the findings presented in Chapter 4 in relation to the research question and the existing literature. Section 5.1 addresses the quantitative findings. Section 5.2 considers the qualitative themes. Section 5.3 acknowledges limitations.

Quantitative Results

The improvement in accuracy of 12.3 percentage points is consistent with the theoretical prediction established in Chapter 3. This finding aligns with the work of (Sánchez et al. 2025), who demonstrated comparable gains in the context of database query optimisation.

Qualitative Themes

The qualitative finding that participants valued clarity and integration over raw performance suggests that usability is a critical factor in the adoption of such frameworks. This dimension was not captured by the accuracy metric and represents an avenue for further investigation.

Limitations

The primary limitation of this study is the use of a convenience sample, which may reduce the generalisability of the findings to other populations. The controlled laboratory setting may not fully reflect the complexity of real-world deployments.

CONCLUSIONS

This chapter summarises the project, addresses each objective, and proposes directions for future work.

The aim of this dissertation was to investigate whether a logic-based analytical framework could improve accuracy in information systems analysis. Three objectives were established (Chapter 1); all three have been fully addressed.

Objective 1 — to critically review existing frameworks — was addressed in Chapter 2, which identified formal logic as a well-established but underexplored foundation for analytical tools.

Objective 2 — to develop a novel framework — was addressed in Chapter 3, where the design and operationalisation of the proposed approach were described in detail.

Objective 3 — to evaluate the framework empirically — was addressed in Chapters 4 and 5. The proposed method achieved 84.7% mean accuracy compared with 72.4% for the baseline.

The hypothesis — that the logic-based framework would yield measurably higher accuracy — is supported by the experimental evidence. The null hypothesis is therefore rejected at the conventional significance level.

The principal limitation of the study is the sample size and setting; future research should replicate the findings with larger, more diverse participant groups in operational environments.

It is concluded that formal logic offers a viable and demonstrably effective foundation for analytical frameworks in information systems, and that the approach merits further development and deployment.

BIBLIOGRAPHY

- Agent Skills. 2026. “Agent Skills Overview.”. <https://agentskills.io/>.
- Clyburn, Cedric. 2026. “MCP Servers Vs. Skills: Choosing the Right Context for Your Ai.” May. <https://developers.redhat.com/articles/2026/05/25/mcp-servers-vs-skills-choosing-right-context-your-ai>.
- Debenedetti, Edoardo, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. “Agentdojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.” In “Advances in Neural Information Processing Systems (Neurips).” Special issue, *Advances in Neural Information Processing Systems (Neurips)*,. <https://doi.org/10.5281/zenodo.12528188>.
- Fraser, Katie, and Tobias Lindenbauer. 2025. “Cutting Through the Noise: Smarter Context Management for LLM-Powered Agents.” December. <https://blog.jetbrains.com/research/2025/12/efficient-context-management>.
- Gulyamov, Saidakhror, Said Gulyamov, Andrey Rodionov, et al. 2026. “Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, And Defense Mechanisms.” *Information* 17 (54). <https://doi.org/10.3390/info17010054>.
- Gutowska, Anna. 2024. “What Are AI Agents?.” July. <https://www.ibm.com/think/topics/ai-agents>.
- Jhandi, Polaris, Owais Kazi, Shreyas Subramanian, and Neel Sendas. 2026. “Small Language Models for Efficient Agentic Tool Calling: Outperforming Large Models with Targeted Fine-Tuning.” In “Proceedings of the Association for the Advancement of Artificial Intelligence (Aaai).” Special issue, *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)* (Seattle, WA, USA),. <https://arxiv.org/abs/2512.15943>.
- Lewis, Charlotte. 2026. “How UK Smes Are Using AI in 2026.” February. <https://www.airit.co.uk/insights-resources/how-uk-smes-are-using-ai-in-2026>.
- Ode, Egena. 2026. “AI Adoption in Greater Manchester Smes – Challenges and Opportunities.” February. https://www.mmu.ac.uk/sites/default/files/2026-03/AI_In_GM_Report_Digital_Adoption.pdf.
- Raghunathan, Sowmya. 2025. “Agents Vs Skills Vs MCP: A Practical Guide to Building Your AI Stack in 2026.” December. <https://www.linkedin.com/pulse/agents-vs-skills-mcp-practical-guide-building-your-ai-raghunathan-oawhc>.
- Sánchez, Esther, Reyes Calderón, and Francisco Herrera. 2025. “Artificial Intelligence Adoption in Smes: Survey Based on TOE–DOI Framework, Primary Methodology and Challenges.” *Applied Sciences* 15 (12): 6465. <https://doi.org/10.3390/app15126465>.
- Unsloth. 2026a. “Fine-Tuning Llms Guide | Unsloth Documentation.”. <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide#fine-tuning-misconceptions>.

- Unsloth. 2026b. “Gemma 4 - How to Run Locally | Unsloth Documentation.”. <https://unsloth.ai/docs/models/gemma-4?q=gemma#run-gemma-4-tutorials>.
- Whittaker, Phil. 2026. “MCP Vs Agent Skills: Why They're Different, Not Competing.” February. <https://www.philwhittaker.dev/blog/mcp-vs-agent-skills-why-theyre-different-not-competing>.
- Zhan, Qiusi, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. “INJECAGENT: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents.” In “Proceedings of the Association for Computational Linguistics (Acl).” Special issue, *Proceedings of the Association for Computational Linguistics (ACL)*,. <https://arxiv.org/abs/2403.02691>.
- Zhang, Dongsun, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. 2025. “MCP SECURITY BENCH (MSB): BENCHMARKING ATTACKS AGAINST MODEL CONTEXT PROTOCOL IN LLM Agents.” *Arxiv Preprint*,. <https://arxiv.org/abs/2510.15994>.
- Zupan, Mario. 2025. “Developing an Accounting Virtual Assistant Through Supervised Fine-Tuning (SFT) of a Small Language Model (Slm).” *Intelligent Systems in Accounting, Finance and Management*,. <https://onlinelibrary.wiley.com/doi/full/10.1002/isaf.70011>.

GLOSSARY AND ABBREVIATIONS

IS	Information Systems
IPI	Indirect Prompt Injection
ML	Machine Learning
SMEs	Small and Medium-sized Enterprises
LoRA	Low-Rank Adaptation
MCP	Model Context Protocol
SLMs	Small Language Models
LLMs	Large Language Models
AI	Artificial Intelligence
SFT	Supervised Fine-Tuning
PEFT	Parameter-Efficient Fine-Tuning
RAG	Retrieval-Augmented Generation
L1	Level 1
L2	Level 2
TOE	Technology-Organisation-Environment
DOI	Diffusion of Innovations
AI Agent	An autonomous system defined by stateful cognitive loops (perception, reasoning, action, and memory). It dynamically adapts its execution path based on real-time feedback from its environment rather than following a rigid, predefined script.(Gutowska 2024)

APPENDIX A: SURVEY INSTRUMENT

The following questionnaire was used to collect primary data during the experimental phase. Each item was rated on a five-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).

A.1 Section 1: Usability

1. The output of the analytical framework was easy to understand.
2. The framework integrated well with the existing tools in use.
3. Using the framework reduced the mental effort required during analysis.
4. It would be recommended to a colleague working in a similar context.

A.2 Section 2: Performance

1. The framework produced results more quickly than the previous approach.
2. The accuracy of results was satisfactory for the task at hand.